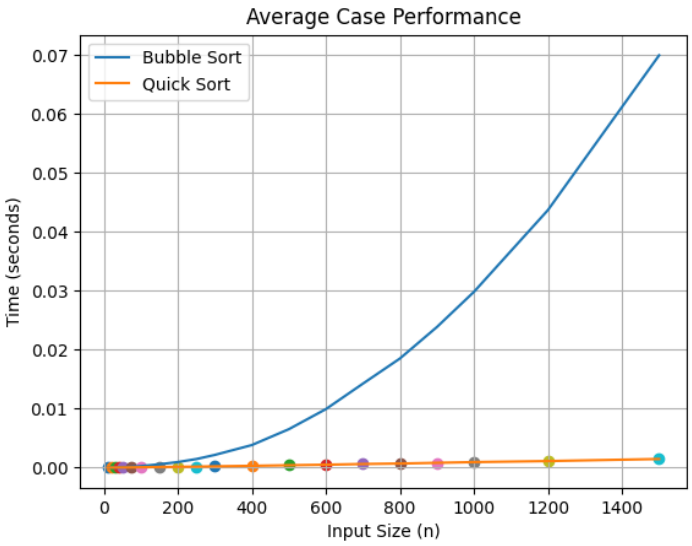
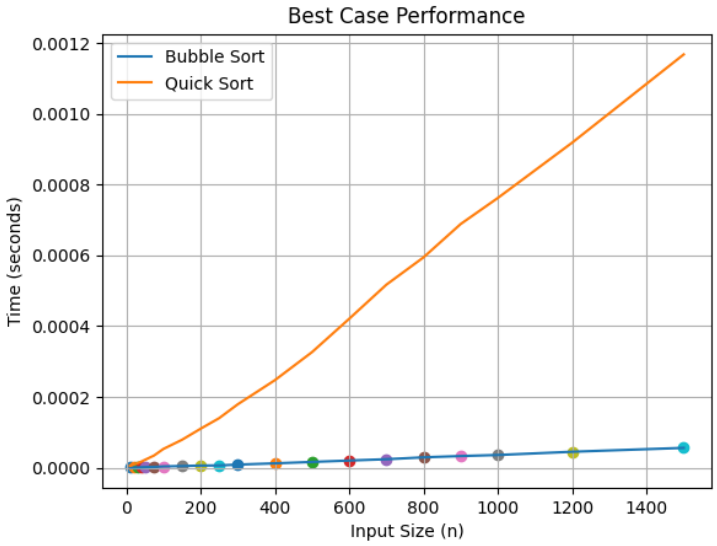
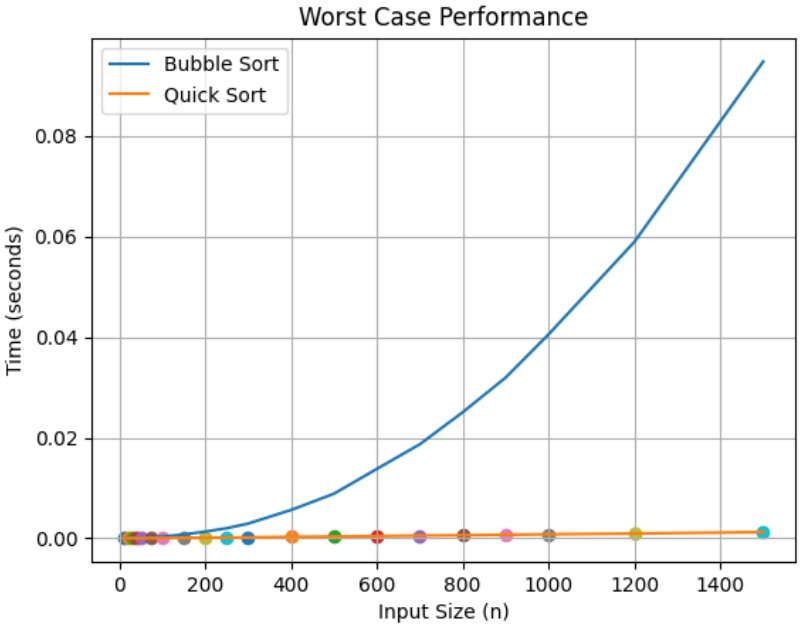

Average Case Performance
Bubble Sort
Quick Sort
Time (seconds)
Input Size (n)
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0
200
400
600
800
1000
1200
1400


Best Case Performance
Bubble Sort
Quick Sort
Time (seconds)
Input Size (n)
0.0000
0.0002
0.0004
0.0006
0.0008
0.0010
0.0012
0
200
400
600
800
1000
1200
1400


Worst Case Performance
Bubble Sort
Quick Sort
Time (seconds)
Input Size (n)
0.00
0.02
0.04
0.06
0.08
0
200
400
600
800
1000
1200
1400

Q3:

1. Best Case Performance

Input: Already sorted array

In the best case, Bubble Sort (with early termination optimization) runs in **O(n)** time because it detects that no swaps are required and terminates after one pass.

Quick Sort, however, still performs recursive partitioning, resulting in approximately **O(n log n)** time complexity.

Observation from Plot

- For small input sizes, both algorithms perform similarly.
- As input size increases, Bubble Sort may perform slightly better due to its linear behavior in this case.

Conclusion

In the best-case scenario, Bubble Sort can be competitive and may outperform Quick Sort because it avoids unnecessary work when the input is already sorted.

2. Average Case Performance

Input: Randomly generated array

In the average case:

- Bubble Sort runs in $\mathbf{O(n^2)}$ time.
- Quick Sort runs in **O(n log n)** time.

Observation from Plot

- For very small input sizes, the difference in performance is minimal.
- After a certain threshold (approximately $n \approx 50–75$), Quick Sort clearly outperforms Bubble Sort.
- Bubble Sort's runtime increases rapidly due to quadratic growth.

- Quick Sort scales much more efficiently.

Conclusion

In the average case, Quick Sort significantly outperforms Bubble Sort for medium and large input sizes. The quadratic complexity of Bubble Sort makes it inefficient as n grows.

3. Worst Case Performance

Input: Reverse sorted array (with last-element pivot selection)

In the worst case:

- Bubble Sort runs in **$O(n^2)$** time.
- Quick Sort also degrades to **$O(n^2)$** time when poor pivot selection is used.

Observation from Plot

- Both algorithms show quadratic growth.
- Their performance curves increase at similar rates.
- In some cases, Bubble Sort may appear slightly faster due to lower overhead.

Conclusion

In the worst-case scenario, Quick Sort loses its advantage and performs similarly to Bubble Sort due to its degraded quadratic behavior.

Q4:

Threshold Selection

From the average-case performance plot, quicksort begins to consistently outperform bubble sort at approximately **n = 75**.

For very small inputs, bubble sort performs competitively due to its low overhead and simple control structure. However, as input size increases, the quadratic time complexity of bubble sort causes its runtime to increase rapidly.

Quicksort, with its average-case complexity of O(n log n), scales significantly better and remain much faster for larger inputs.

Therefore, we define arrays of size **n ≤ 75** as “small” and use bubble sort for those inputs. For arrays with **n > 75**, quicksort is selected.